Introduction to Applied Cryptography

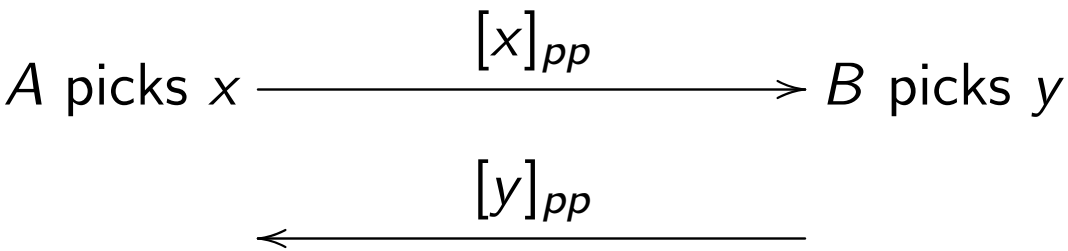
Thomas Peters – F.-X. Standaert

Slides 06



Key Agreement

How do A and B get their keys?



- A and B should be able to compute the same key k from either x and $[y]_{pp}$ or $[x]_{pp}$ and y, say $k := H([xy]_{pp})$
- Given only $[x]_{pp}$ and $[y]_{pp}$, computing $[xy]_{pp}$ must be hard

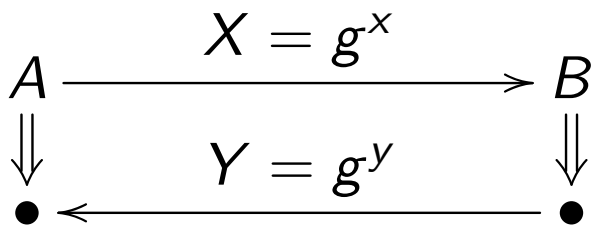
Goal: Building boxes $[\cdot]_{pp}$ so that

- Adding in any box is easy
- Removing from a box or merging two boxes are hard

The Diffie-Hellman Protocol (Outline)

Prime Order Cyclic Groups

Gets $(\mathbb{G}, q, g) \leftarrow \mathcal{G}(1^n)$ with $\langle g \rangle = \mathbb{G}$ and prime $q = |\mathbb{G}| \geq 2^n$



- A and B compute $k := g^{x \cdot y}$ as $(g^y)^x$ or $(g^x)^y$
- Computing $g^{x \cdot y}$ given only X and Y must be hard

Properties:

- Efficiency: square and multiply
- Security: *Computational Diffie-Hellman* assumption

Computational Diffie-Hellman Problem

It should be hard to merge two boxes $[x]_{pp}$ and $[y]_{pp}$!

Consider experiment $\mathrm{CDH}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $(x, y) \xleftarrow{R} \mathbb{Z}_q^2$
3. Set $h \leftarrow \mathcal{A}(\mathbb{G}, q, g, g^x, g^y)$
4. $\mathrm{CDH}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow h = g^{x \cdot y}$

Computational Diffie-Hellman Problem

The *computational Diffie-Hellman problem* is hard relative to $\mathcal{G}$ if $\forall$ PPT $\mathcal{A}$, there is a negl. ϵ s.t.:

$$\Pr[\mathsf{CDH}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \epsilon(n)$$

Observe:

- If $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}$ is easy then $\mathsf{CDH}_{\mathcal{A},\mathcal{G}}$ is easy
- The converse is not necessarily true!
- The CDH assumption states that the CDH problem is hard

Decisional Diffie-Hellman Problem

We can even want that $[xy]_{pp}$ looks random

Consider experiment $\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $(x, y, z) \xleftarrow{R} \mathbb{Z}_q^3$
3. Flip a coin $b \xleftarrow{R} \{0,1\}$, and set $h_1 := g^{x \cdot y}$ and $h_0 := g^z$
4. Set $b' \leftarrow \mathcal{A}\big(\mathbb{G}, q, g, (g^x, g^y, h_b)\big)$
5. $\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow b = b'$

Observe:

- *Decide* whether h_b is a CDH solution g^{xy} or a random g^z



Decisional Diffie-Hellman Problem

The *decisional Diffie-Hellman problem* is hard relative to $\mathcal{G}$ if $\forall$ PPT $\mathcal{A}$, there is a negl. $\epsilon(n)$ s.t.:

$$\Pr[\mathsf{DDH}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \frac{1}{2} + \epsilon(n),$$

Observe:

- If $\mathsf{CDH}_{\mathcal{A},\mathcal{G}}$ is easy then $\mathsf{DDH}_{\mathcal{A},\mathcal{G}}$ is easy
- The converse is believed to be false
- The DDH assumption states that the DDH problem is hard

What can go wrong with DH-KA?

No hash

- For instance, use the first 256 bits of g^{xy} as k
- Need $H(g^{xy})$ with a Key Derivation Function (KDF)
- KDF: HKDF, HMAC-based KDF or SHA-3

Too many (small) subgroups

- OpenSSL vulnerability fixed (only!) in 2016
- Worst case $p = 2q_1 \cdots q_\ell + 1$ with small primes $q_1, \ldots, q_\ell$

Non-authenticated KA

- "Anonymous" DH (man-in-the middle attack)

HKDF with HMAC-SHA256

Input: secret S (from a good source)
Output: bitstring K of desired length *size*
HKDF$(salt, S, info, size) = K$

Extract

$$PRK = \text{HMAC-SHA256}(salt, S, size)$$

Expand

$$H_1 = \text{HMAC-SHA256}(PRK, info, \langle 1 \rangle)$$
$$H_2 = \text{HMAC-SHA256}(PRK, H_1 || info, \langle 2 \rangle)$$
$$\vdots$$
$$H_\ell = \text{HMAC-SHA256}(PRK, H_{\ell-1} || info, \langle \ell \rangle)$$

Until $K := H_1 || \cdots || H_\ell$ is such that $|K| = size$

HMAC

Input: secret key k and message M
Output: tag T (of size of the one of the compression function)

$$T = H\big((k' \oplus opad) \,||\, H((k' \oplus ipad) \,||\, M) \big)$$

where $k' = H(k)$ if k larger than block-size, else $k' = k$.

With SHA-256:

How to define $\mathbb{G}$ from $\mathbb{Z}_p^*$

Good candidate:

$|p| = 3072$ and $|q| = 256$

1. Find primes p and q such that $p = 2kq + 1$ for some k.
2. Pick a random g_0 over $\mathbb{Z}_p^*$ and compute $g = g_0^{2k} \bmod p$
3. If $g \neq 1$, set $\mathbb{G} = \langle g \rangle$, else, go back to 2.

Recall: $\forall a \in \mathbb{Z}_p^*$

- $a^{p-1} = 1 \bmod p$ (Fermat)
- $\mathrm{ord}(a)$ is the smallest non-negative i such that $a^i = 1 \bmod p$
  then, $i | p - 1$ and $a^x = a^{x \bmod i} \bmod p$

The Discrete Logarithm Problem

Removing from the box $[\cdot]_{pp}$ should be hard

Consider experiment $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $h \xleftarrow{R} \mathbb{G}$
3. Set $x \leftarrow \mathcal{A}(\mathbb{G}, q, g, h)$
4. $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow g^x = h$

The *discrete logarithm problem* is hard relative to $\mathcal{G}$ if
$\forall$ PPT $\mathcal{A}$, there is a negligible ϵ s.t.:

$$\Pr[\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \epsilon(n)$$

Elliptic Curves Cryptography (ECC)

Shorter keys: ECC 256 bits vs. $\mathbb{Z}_p^*$ 3072 bits

Groups defined on the points of elliptic curves

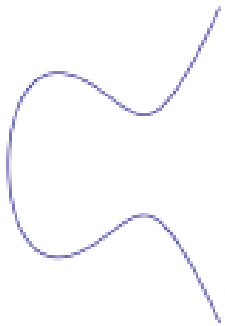
$$y^2 = x^3 + Ax + B \mod p$$

$E(\mathbb{Z}_p) := \{(x, y) \in \mathbb{Z}_p \times \mathbb{Z}_p \text{ on the curve } \} \cup \mathcal{O}$

Examples:



$y^2 = x^3 - 2x$



$y^2 = x^3 - 2x + 2$

Diffie-Hellman Key Agreement

Usage in TLS: [1]

	Key exchange/agreement and authentication					
Algorithm	**SSL 2.0**	**SSL 3.0**	**TLS 1.0**	**TLS 1.1**	**TLS 1.2**	**TLS 1.3**
RSA	Yes	Yes	Yes	Yes	Yes	No
DH-RSA	No	Yes	Yes	Yes	Yes	No
DHE-RSA (forward secrecy)	No	Yes	Yes	Yes	Yes	Yes
ECDH-RSA	No	No	Yes	Yes	Yes	No
ECDHE-RSA (forward secrecy)	No	No	Yes	Yes	Yes	Yes
DH-DSS	No	Yes	Yes	Yes	Yes	No
DHE-DSS (forward secrecy)	No	Yes	Yes	Yes	Yes	No[43]
ECDH-ECDSA	No	No	Yes	Yes	Yes	No
ECDHE-ECDSA (forward secrecy)	No	No	Yes	Yes	Yes	Yes

[1] Credit: Wikipedia



Authenticated Key Agreement (AKA)

Diffie-Hellman AKA

Given $(\mathbb{G}, q, g) \leftarrow \mathcal{G}(1^n)$ and public verification keys pk_A and pk_B

$\mathcal{A}$ knows sk_A		$\mathcal{B}$ knows sk_B
$\sigma_A = \mathsf{Sign}(\mathsf{sk_A}, \mathsf{X})$	$\xrightarrow{\sigma_A, X = g^x}$	$\sigma_B = \mathsf{Sign}(sk_B, Y)$
$\mathsf{Verif}(pk_B, Y, \sigma_B) \stackrel{?}{=} 1$	$\xleftarrow{\sigma_B, Y = g^y}$	$\mathsf{Verify}(pk_A, X, \sigma_A) \stackrel{?}{=} 1$
$KDF(Y^x)$		$KDF(X^y)$

Futher Attacks

Impersonation in case of data leak

- That is, ephemeral secret $\textit{eph-key} = x$ leaks
- Allows a replay attack with DH-AKA

Additional protection

- Naxos trick: derive exponent $x = H(\textit{eph-key}, \textit{long-term-key})$
- Idea: both types of keys are stored and protected differently



"Self-Authenticated" Diffie-Hellman Key Agreement

3-DH as used in Element

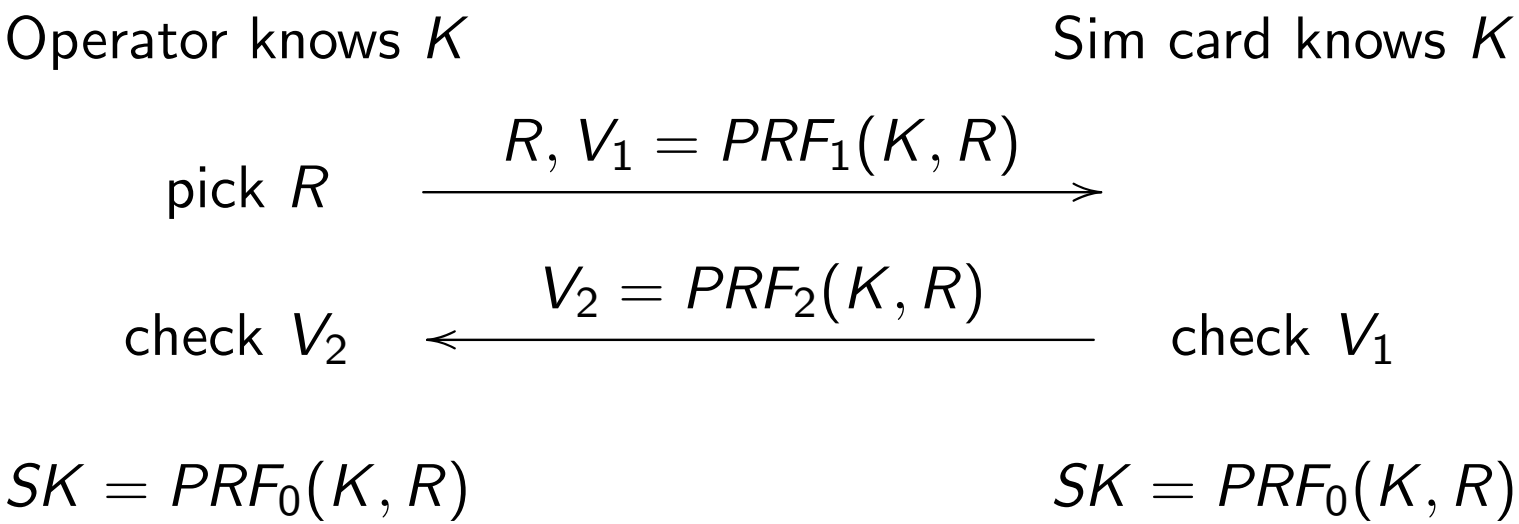
Given $(\mathbb{G}, q, g) \leftarrow \mathcal{G}(1^n)$ and public $A = g^a$ and $B = g^b$

$\mathcal{A}$ knows a		$\mathcal{B}$ knows b
picks x	$\xrightarrow{X = g^x}$	picks y
computes	$\xleftarrow{Y = g^y}$	computes
$KDF(Y^x, B^x, Y^a, view)$		$KDF(X^y, X^b, A^y, view)$

where $view = X, Y, A, B$

Non-DH Authenticated Key Agreement

4G/5G communications (symmetric AKA!)



Purpose: not using K all the time, used for other things as well.
Breach (long-term key) allows man-in-the-middle attack
Not forward secure by keeping the seen R's
Key control: Sim does not participate to the choice of SK

Secure Messaging

End-To-End (E2E) Encryption

- Only the sender and the intended reciever get messages
- Neither the operator, nor any intermediate (servers)...
- No third-party key-escrow (E2E ~~but~~...)

Key-Exposure Resilience

- Forward secrecy, but depends on refreshed keys
- How often keys should be refreshed?
- (Double) Ratchet mechanism (post key-compromission)

